Pruning Paltry Predictions: A Speculative Decoding Inference Engine

Tobias Moser Eli Wandless

1. Problem Statement

In this project, we implement a system for efficient LLM inference on a multi-GPU cluster, concentrating on the inference framework known as speculative decoding. Our project consisted of three main components: a set of model and hardware-optimized CUDA kernels, a host-level speculative decoding harness, and a inter-GPU MPI orchestrator. Specifically, we used Qwen 2.5 7B as the target model and Qwen 2.5 0.5B as the draft model, and ran all inference using the Quadro RTX 6000 GPUs in the CME 213 cluster. Our ultimate goal was to achieve the best performance possible given the model, hardware, and time constraints of the project. The experimental structure took the form of a standard inference setup in which inputs were tokenized natural language prompts, and outputs were sequences of model-generated output tokens. For simplification, and to best capture the speeds a single user would experience with our system, all benchmarking and evaluation was done using batch sizes of 1. This problem space is extremely important in modern research and enterprise contexts, many of which rely of on high-throughput LLM usage for tasks like data analysis and code generation. Speculative decoding naturally requires a multi-GPU system, as the memory footprints of large target and draft models typically exceed the limits of a single GPU. Additionally, a mutli-GPU configuration naturally scales as one increases the number of batches and draft models being used.

2. Algorithms and State of the Art

The high-level algorithm used in our inference engine is speculative decoding (Leviathan et al., 2023). The section below describes the algorithm as it was presented in the original paper. Lower kernel-level algorithms are described in Sections 3.2 and 7.

2.1. Overview of Speculative Decoding

Let M_p be the target model (i.e. the model whose inference we are trying to accelerate), and $p(x_t|x_{<t})$ the distribution we get from the model for a token prefix $x_{<t}$. Let M_q be the draft model (i.e. a more efficient model that approximates the target model's outputs for a given task), and denote by $q(x_t|x_{<t})$ the distribution we get from the model for a prefix $x_{<t}$. Speculative decoding uses the more efficient model M_q to generate γ completions, then uses the target model M_p to evaluate all the drafted tokens M_q in parallel, accepting tokens that lead to an identical distribution. This process allows M_p to sample one additional bonus token from distribution over the last *accepted* token, guaranteeing each step of the target model M_p produces at least one new token. It follows that the target model never makes more total forward passes than it would with standard autoregressive sampling, and generates up to $\gamma + 1$ tokens per forward pass depending on how well M_q approximates M_p.

2.2. Speculative Sampling

First, note that speculative decoding functions irrespective of the specific sampling regime (i.e. greedy, stochastic, etc), so let us define $p(x)$ and $q(x)$ as the distributions from M_p and M_q respectively, adjusted for the sampling method. To sample $x \sim p(x)$, we instead sample $x \sim q(x)$, keeping it if $q(x) \leq p(x)$. If $q(x) > p(x)$, we reject the sample with probability $1 - \frac{p(x)}{q(x)}$, and sample x again from an adjusted distribution $p'(x) = norm(max(0, p(x) - q(x)))$. In their original paper, (Leviathan et al., 2023) show that this method provably results in a probability distribution identical to the original $p(x)$. See Appendix section A.1 below for the speculative decoding in algorithmic form.

2.3. Alternative Algorithms

Alternative host algorithms aim to improve verification efficiency or to eliminate the draft model. SpecInfer (Miao et al., 2024) uses tree-based speculative verification, which structures multiple speculative execution paths into a tree and thereby enables the target model to verify multiple candidate token sequences simultaneously in a single batched forward pass. Medusa (Cai et al., 2024) removes the draft model by attaching parallel decoding heads to the target model. Each head predicts tokens at a different future position (e.g., $t + 1, t + 2, \dots$), and the host verifies these predictions in a single target pass using a tree-based validation schema. Because the standard speculative decoding implementation we used is nominally less efficient than these more advanced techniques – SpecInfer, for instance, can produce 1.5x to 3.5x speedups over regular speculative decoding – we had hoped to extend our implementation but were unable to do so due to time constraints.

3. Parallelization

3.1. Draft Target Split

At a high level, the target model weights are held on rank 0 and draft model's on rank 1. The draft model produces γ tokens before using MPI's `send` and `recv` primitives to pass these tokens to the target model. The target model accepts some subset of these tokens following the process described in Section 2, and passes this information back to the draft model. The draft model rolls back its KV cache accordingly, overwriting entries corresponding to rejected tokens, and repeats this process. This distribution of labor across GPUs was the natural and industry-standard way of handling speculative decoding, so we didn't feel it was necessary to experiment with radically different setups. Additionally, the very nature of speculative decoding requires a simplistic MPI environment, as all communication is linear point-to-point sends and receives.

3.2. Inference Kernels

At a lower level, our implementations of the draft and target models run on custom CUDA kernels tuned to the Quadro RTX 6000 hardware. The embedding layer is a vectorized row-gather over the embedding weights. Normalization layers have blocks reduce along the hidden dimension to compute a mean of squares, followed by a fused normalize and scale. The the attention and SwiGLU GEMMs are dispatched as cuBLAS Tensor-Core calls accumulating in `fp32`. The SwiGLU activation and elementwise multiply are fused into a single custom kernel. Attention uses a FlashAttention2 style kernel with `nvcuda::wmma` fragments for the QK^T and PV GEMMs during prefill, and a separate flash-decoding kernel that blocks along KV history for the decode phase. RoPE is fused into the KV-cache write and flash-attention kernels. Where possible, HBM accesses use 128-bit vectorized loads for coalescing and fewer instructions. More detailed kernel implementation details are provided in Appendix A.2. All kernel designs were arrived at by iteratively implementing a kernel, profiling it to identify bottlenecks, then re-writing the code to address these deficiencies.

Component	Qwen2.5-7B	Qwen2.5-0.5B
Layers	28	24
Hidden dimension	3,584	896
Attention heads	28Q, 4KV	14Q, 2KV
Head dimension	128	64
Intermediate Size	18,944	4,864
Positional encoding	RoPE	RoPE
Normalization	RMSNorm	RMSNorm
Activation	SwiGLU	SwiGLU

Table 1. Architectures of Qwen2.5-7B and Qwen2.5-0.5B

4. Performance Modeling

To predict and interpret our results, we model the theoretical performance of our systems on the Quadro RTX 6000 GPU, which offers a peak memory bandwidth of $\text{BW}_{\text{peak}} \approx 672$GB/s (of which $\sim$80%, or $\text{BW}_{\text{eff}} \approx 540$ GB/s, is realistically achievable), and a peak `fp32` tensor-core throughput of ≈ 65TFLOP/s. It follows that the roofline ridge point is $\frac{65*10^{12}}{672*10^9} \approx 96$ FLOPs/byte. Any workload or kernel with arithmetic intensity below this is bound by the systems memory bandwidth. There are three relevant phases to our workload that we model below.

4.1. Prefill

The prefill stage loads every model weight onto the chip to process the initial prompt, irrespective of its length S. making prefill compute-bound once S exceeds the ridge point of ~ 96. For simplicity, we assume $S \geq 96$, since speculative decoding is aimed at accelerating the decode phase.

4.2. Decode and Verify

Decode is the steady state in which a model reads every weight from HBM to autoregressively produce one token; verify is the step in which the target checks $\gamma + 1$ draft tokens at once. Since $\gamma \leq 8$, both are bottlenecked by streaming all model weights from memory with essentially the same latency t_{token}, which we estimate as memory traffic over achievable bandwidth BW:

$$t_{\text{token}} \approx \frac{W_{\text{stream}}}{\text{BW}}, \qquad (\text{tok/s})_{\max} \approx \frac{\text{BW}}{W_{\text{stream}}}, \qquad (1)$$

where W_{stream} is the 7 billion model weights $\times 2$ bytes per `fp16` streamed from HBM (14.1 GB target, 0.99 GB draft).

By Little's law, sustaining bandwidth B over the Quadro RTX 6000's memory latency $L \approx 350$ ns requires $Q = B \cdot L \approx 270$ KB of loads in flight. The target model is large enough to saturate the bus. However, the draft model is small enough that HBM access *operations* are completed before subsequent *instructions* can be issued, sustaining roughly half of Q^*. We therefore expect the draft model to reach only half its ceiling. Table 2 gives both the theoretical ceiling and this Little's-law adjusted estimate.

Model	Stream	Ceiling (tok/s)	Little's law (tok/s)
Target	14.1 GB	38	$\sim$34
Draft	0.99 GB	550	$\sim$310

Table 2. Theoretical Decode throughput: the bandwidth-saturated ceiling versus the Little's-law estimate. ($\sim 0.7\,Q^*$ for the wide target model, ~ 0.45 for the draft).

Decode latency rises slightly with context length, since the

attention reads the whole KV cache (57.3 KB/token for the target) in addition to the weights. At $S = 2048$ this is 117 MB, just 0.8% of the weight stream, so we treat it as negligible for our $S \leq 4096$ workloads.

Falsifiable predictions. We expect target decode latency to be within $\sim$15% of its ceiling, while the draft to reach only $\sim$half of its. We expect decode latency to stay nearly flat in S, rising gradually with the linear KV read. Similarly, we expect verify latency to remain flat in γ, and have the same latency as one decode step. Results are reported in Section 6.

4.3. Speculative Decoding Modeling

We assume draft tokens are accepted independently at a rate α (in reality α varies with token and context, but this is the simplification made by Leviathan et al). In section 4.2, we show that the target's verification of $\gamma + 1$ candidate tokens costs approximately the same as single token decode. Each draft token costs $c \cdot T_{\text{target}}$, where $c = T_{\text{draft}}/T_{\text{target}} \approx 0.99/14.1 \approx 0.07$ from the weight-stream ratio of Section 4.2. Following Leviathan et al., the expected number of tokens produced and the wall-time per round are

$$\mathbb{E}[\text{tokens}] = \frac{1-\alpha^{\gamma+1}}{1-\alpha}, \qquad T_{\text{round}} \approx T_{\text{target}}\,(1+\gamma c)\,. \tag{2}$$

Since target-only decode produces one token in T_{target}, the predicted end-to-end speedup is

$$\text{Speedup}(\gamma) = \frac{1-\alpha^{\gamma+1}}{(1-\alpha)\,(1+\gamma c)}. \tag{3}$$

Table 3 evaluates Eq. (3) at $c = 0.07$. The optimal γ grows with the acceptance rate and the peak attainable speedup is bounded by $1/(1-\alpha)$ (the $\gamma \to \infty$, $c \to 0$ limit). We note that the optimal γ falls near 5–7, consistent with our kernel's compile-time cap of $\gamma \leq 8$.

	γ					
α	1	2	3	4	5	6
0.6	1.50	1.72	**1.80**	1.80	1.77	1.72
0.7	1.59	1.90	2.09	2.17	**2.18**	2.15
0.8	1.68	2.08	2.38	2.59	2.73	**2.78**

Table 3. Predicted speedup from Eq. (3) at $c = 0.07$. Bold marks the optimal γ for a given acceptance rate α.

Falsifiable predictions. We expect the speedup over target-only decode, given an empirically measured acceptance rate α, should match the corresponding row of Table 3. Sweeping γ should reveal a speedup maximum at the predicted γ^*, and speedup should degrade toward $1\times$ as $\alpha \to 0$.

5. Benchmarking and Instrumentation

We detail the methods and metrics we used for benchmarking in this section. Broadly speaking, our performance models from Section 4 accurately predicted performance for the individual draft and target models, though end-to-end speculative decoding performance was ultimately bottlenecked by the acceptance rate α so we couldn't perfectly observe the speedup laws from Table 2. In Section 6, we scrutinize how they compare to our predicted performance in more detail.

5.1. Kernels

We benchmark our at several levels of the inference hierarchy. During development, modules directly ported from HuggingFace's `transformers` library, (e.g. `Qwen2RMSNorm`) are used as our reference and benchmark our custom kernel on a set of different (B, S) workloads. We benchmark timing against HuggingFace runs with and without using `torch.compile`.

For profiling, a separate path launches our kernel(s) wrapped in an NVTX range and surrounded by `cudaProfilerStart/Stop`. This enables two levels of analysis. Individual kernel metrics are reported in an `.ncu` replay, and our per-kernel optimizations were often informed by the utilization, memory utilization, occupancy, and warp scheduling data that these expose. At a higher level, we analyze multi-kernel workloads like transformer layers or entire forward passes with `.nsys` artifacts via Nsight systems. This helped identify what kernels dominate runtime, and where we were bottlenecked by kernel launches.

5.2. MPI

We benchmarked blocking MPI transfer latency between the draft and target GPUs within a single node. The dominant cost is the draft-to-target payload (γ full vocabulary distributions, $\sim$1.45 MiB at $\gamma = 4$), while the target-to-draft synchronization is negligible (16 bytes). A full speculative round incurs a mean communication latency of **1.51 ms**—roughly 5% of the $\sim$30 ms target forward pass. This confirms that MPI does not bottleneck the pipeline. A detailed latency breakdown by sub-stage is provided in Appendix B.

6. Bottleneck Analysis

6.1. Target 7B

As reported in Section 4.2, the target model streams its full 14.1 GB of weights every forward pass. Table 4 shows that we achieve (34.0 tok/s) at $S \geq 128$, which is 14.1 GB/29.43 ms $\approx$ 479 GB/s — 89% of BW_{eff}. As it achieves near-peak effective bandwidth, the model is band-

	Target 7B		Draft 0.5B	
S	Prefill	Decode	Prefill	Decode
64	1479	33.2	8183	303.7
128	2691	33.1	16166	301.0
256	3961	33.0	31993	299.2
512	4641	32.7	38960	291.8
1024	4697	32.2	42481	283.5
2048	4490	31.2	37744	264.5

Table 4. Full-forward throughput sweep, all values in tok/s (S = prompt length).

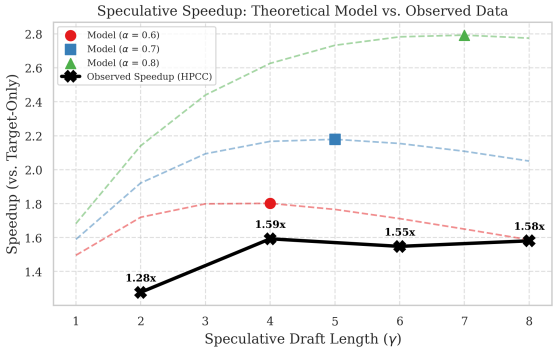

Figure 1. Measured end-to-end speculative speedup over target-only decode as a function of draft length γ (solid), overlaid on the single-α predictions of Eq. (3) (dashed).

width bound and performance gains would require shrinking the memory footprint itself. As predicted, decode latency gradually increase with S, the small linear KV-read slope predicted in Section 4.2.

6.2. Draft 0.5B

The draft model streams 0.99 GB/token, and we measure (299 tok/s $\approx$ 296 GB/s, 55% of BW_{eff}; Table 4). The draft model is bottlenecked by it's ability to saturate the memory bus with sufficient bytes in flight. As predicted in Section 4.2, the draft's per-kernel workloads fall below the $Q^* \approx 270$ KB necessary, so the warps spend time filling and draining the pipeline rather than moving bytes. The measured 299 tok/s matches the $\sim$310 tok/s Little's-law estimate of Table 2, not the saturated ceiling. The KV slope is also relatively steeper than the target's (3.31 $\rightarrow$ 3.78 ms, +14%, over $S = 64 \rightarrow 2048$), because the linear cache read is a larger fraction of the draft's total memory footprint.

6.3. End-to-end Speculative Decoding

Dual-GPU speculative decoding averages $\sim$53 tok/s at $\gamma \in \{4, 6, 8\}$ against the $\sim$33 tok/s target-only baseline, a 1.6$\times$ mean speedup, peaking at 74 tok/s. In this setup, throughput scales with α. High-acceptance categories approach the model's $1/(1-\alpha)$ ceiling, while low-acceptance prose barely clears baseline target-only inference. Dotted lines demonstrate how the optimal γ scales with α, and our solid line demonstrates our measured speedup on a heterogenous set of prompts. End-to-end speedup is therefore bottlenecked by the draft model's acceptance rate, dictated by how well the 0.5B approximates the 7B, rather than any hardware limit. Figure 1 illustrates the measured speedups against the single-α predictions of Eq. (3). The data reproduce the model's qualitative shape: speedup climbs with γ, peaks at 1.59$\times$ near $\gamma = 4$, and then plateaus, confirming both that a finite optimal γ^* exists and that it is small. Because we measure this with a heterogenous prompt mix, the curve reproduces the predicted concave shape, but sits below the $\alpha = 0.6$ ceiling.

7. Algorithmic Variants

For attention, we use Flash Attention 2 (Dao et al., 2022) in prefill and Flash Decoding (Dao et al., 2023) in the decode and verify steps, alongside standard kernel fusion (Wang et al., 2010), vectorized loads (Luitjens), and cuBLAS GEMMs (NVIDIA Corporation, 2020); each kernel is detailed in Section A.2. This section outlines several specific optimizations we made that addresssed a certain bottleneck we identified using Nvidia's Nsight Compute.

7.1. Fused RoPE

Initially, a standalone elementwise kernel read Q and K vectors from HBM, encode their position with RoPE (Su et al., 2023), and wrote them back to HBM. Because they're used in Attention, all QK vectors made two full HBM round-trips. We recognized this redundancy while profiling, and addressed it by fusing the K rotation into our KV-cache write kernel, and the Q rotation into the flash-attention kernel (applied to Q tiles in shared memory before the QK^T GEMM). This optimization removes the standalone RoPE kernel (41.95 μ), and the KV-write rises only from 7.87 to 18.69 μ, dropping from 49.8 μs across three launches to 18.7 μs in a single launch (a 2.7$\times$ reduction). The flash-attention kernel itself falls from 184.8 to 163.1 μs.

7.2. Flash-Decoding

Initially, we used the same flash attention kernel for prefill and decode. This kernel dispatched thread blocks along $(S, \text{query\_heads})$, despite the decode phase operating on $(S = 1)$. The kernel still reads from a long, growing KV history, so this layout left GPU SMs severely underutilized. We measured poor performance at realistic context lengths: 168.6 μs at a 1024-token history (0.88$\times$ eager), 816.4 μs at 4096 (0.57$\times$), and 3233.8 μs at 16,383 (0.69$\times$). Switching to Flash-Decoding, which additionally dispatches thread

blocks along the KV cache, dropped the same workloads to 53.2, 187.6, and 705.5 μs, giving a 3–4$\times$ improvement over standard flash attention.

7.3. CUDA-graph Capture.

The 0.5B draft model streams $\sim$1 GB of weights over 150+ kernel calls. In Nsight systems, we saw the GPU was executing entire kernel's workloads faster than the CPU could launch them, leaving the draft model host-bound. To address this, we capture the entire draft forward pass into a single CUDA graph and replay it per step, collapsing a forward pass into one launch. This lifted draft decode from 167 $\rightarrow$ 299 tok/s, a 1.79$\times$ speedup.

8. Scalability Analysis

To understand our inference engine's performance at scale, we study both strong scaling and weak scaling as functions of the number of target GPU ranks P_{target}. Because of time and compute constraints, we were unable to collect empirical data for strong and weak scaling, and therefore analyze scalability for $P_{\text{target}} \in \{1, 2, 4, 8\}$ using a purely theoretical model (derived from the standard scaling formulas discussed in class) and parameterized by our profiled kernel and network latency constants. The results of this model are plotted in Figure 2.

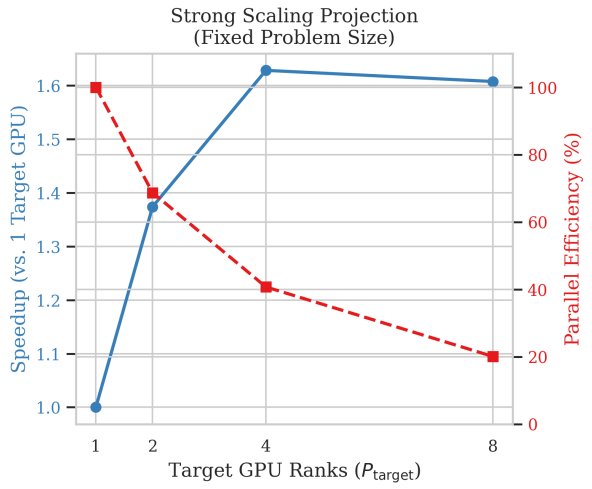

Figure 2. Projected scalability of the speculative decoding engine. Strong scaling speedup and parallel efficiency under a fixed batch size ($B = 1$).

8.1. Strong Scaling Analysis

In the strong scaling regime, we fix the total problem size. We evaluate speculative decoding performance for a single user request (batch size $B = 1$) generating completions with draft length $\gamma = 4$ and a mean acceptance rate of $\alpha = 0.7$ (yielding $\mathbb{E}[\text{tokens}] \approx 2.5$ expected tokens per speculative round), since we have copious real data for this configuration. The target model is sharded across P ranks using tensor parallelism in the standard fashion for decoder models (Liang & Liu, 2024). The total execution time for a single speculative round, T_{round}, is modeled as:

$$T_{\text{round}} = \gamma \cdot t_{\text{draft}} + t_{\text{target\_stream}}(P) + t_{\text{all\_reduce}}(P) + t_{\text{mpi}} \quad (4)$$

where:

- $t_{\text{draft}} = 3.71$ ms is the CUDA Graph-optimized draft decode latency per token.
- $t_{\text{target\_stream}}(P) = \frac{2 \cdot W_{\text{target}}}{P} \cdot \text{BW}_{\text{eff}}$ is the time to stream the sharded target weights ($W_{\text{target}} = 7.07$B parameters, ≈ 14.1 GB) at an effective bandwidth of $\text{BW}_{\text{eff}} \approx 540$ GB/s. For $P = 1$, $t_{\text{target\_stream}} \approx 26.1$ ms.
- $t_{\text{all\_reduce}}(P)$ represents the intra-layer tensor-parallel communication overhead. A single target forward pass requires two All-Reduces per layer (i.e. 56 total for 28 layers). Using our collected MPI benchmark data, we estimate these latencies to be 0.0, 1.5, 3.2, and 6.8 ms for $P \in \{1, 2, 4, 8\}$ respectively.
- $t_{\text{mpi}} = 1.51$ ms is the constant network transfer latency floor for transmitting the 1.45 MiB draft tokens/logits payload and the 16-byte metadata synchronization packet.

The resulting latency per generated token is $t_{\text{token}} = T_{\text{round}}/\mathbb{E}[\text{tokens}]$. The strong speedup is measured relative to speculative decoding running on a single target GPU ($P_{\text{target}} = 1$). As shown in Figure 2 (left), the strong scaling speedup increases from 1.0$\times$ to 1.34$\times$ at $P_{\text{target}} = 2$ and peaks at 1.47$\times$ for $P_{\text{target}} = 4$. Beyond this point, at $P_{\text{target}} = 8$, the speedup degrades back to 1.29$\times$. Amdahl's law models this behavior exactly: as P_{target} scales, the parallelizable portion of the target forward pass ($t_{\text{target\_stream}}$) shrinks from 26.1 ms down to 3.26 ms. However, the serial parts of the workload—namely the sequential drafting phase ($\gamma \cdot t_{\text{draft}} = 14.84$ ms) and the MPI communication floor (1.51 ms)—remain completely unaffected. Concurrently, the tensor-parallel All-Reduce overhead scales up to 6.8 ms. Consequently, parallel efficiency drops precipitously from 100% ($P_{\text{target}} = 1$) to 67% ($P_{\text{target}} = 2$), 37% ($P_{\text{target}} = 4$), and finally 16% ($P_{\text{target}} = 8$). This demonstrates that strong scaling speculative decoding is highly communication-bound and serial-bottlenecked.

8.2. Weak Scaling Analysis

The fundamental unit of work in our speculative decoding engine entails one draft model proposing tokens and one target model verifing them for a single-request batch. To

construct a weak scaling regime, we would have to grow the batch to hold per-rank work constant. This makes weak scaling a somewhat incoherent medium for analyzing our workload.

9. Correctness and Verification

Our experimental setup allowed for extremely simple correctness and verification across all stages of the project. During the initial kernel development stages, we validated our custom CUDA kernels by comparing their results directly against the Pytorch implementation of the corresponding Qwen operation as it was written in the Transformers library. We tested both with randomly initialized input and weight tensors as well as with the actual model weights for all kernels, using the standard Pytorch `all_close` operation with $\epsilon = 0.001$ as the standard for correctness. Once we had all kernels written for both the draft and target models, we first ran a forward pass (with deterministic sampling) for the original two models using the unmodified and standard Transformers inference API to collect the baseline outputs, then patched our kernels into Pytorch via Pybind and reran inference using the same model weights and sampling strategy but now with every operation being executed by our kernels. We then compared the two sets of outputs and ensured they matched perfectly. Finally, we confirmed our implementation of speculative decoding was correct by verifying the responses matched that of the target model with deterministic sampling.

10. Discussion and Future Work

There were a variety of trade-offs we made over the course of this project. First, in order to have a more robust and optimized kernel stack for our engine, we spent a considerable amount of time doing solely CUDA programming. This meant that while the individual base inference operations were highly performant, we spent less time on the higher-level algorithms. In particular, our host-orchestration code is quite simplistic relative to state of the art inference engines like SGLang and vLLM, and we only implemented the most simple version of speculative decoding. Because we were operating on a shared cluster and had to avoid allocating an overly large number of GPUs in a run, we chose models small enough to fit entirely on the VRAM of a single Quadro RTX 6000, and did not experiment with larger, more performant models like Kimi K2 or Deepseek V4 sharded across multiple GPUs.

There are still a number of interesting directions for immediate future work. First, we really wanted to implement a more advanced variant of speculative decoding, like the previously mentioned SpecInfer (Miao et al., 2024) and Medusa (Cai et al., 2024); the former, in particular, stands to benefit the most from our multi-GPU setup and could potentially yield considerable speedups. Second, we would also like to extend the host-level code of our inference engine to simultaneously be more performant and flexible. As it stands, our implementation is optimized for single-batch, long-sequence length inference. However, modern inference engines are designed to effectively manage large and asymmetric batch sizes, as well as batches that arrive across different network links and at different times. To function effectively in such a paradigm, we would need to extend our host code to coordinate across staggered batches; we would likely want to make use of modern optimizations like Paged Attention (Kwon et al., 2023). Finally, we also could experiment further with the distributions and configurations of the target and draft models. For example, we could test whether or not hosting both models on the same GPU reduces MPI overhead enough to create real performance gains without overly damaging our memory capacity. We could similarly experiment with running multiple draft models in parallel (either on a single or on multiple GPUs) to keep the target model fully saturated at all times.

Most of the optimizations we made were worth the effort. The sole exception to this was collecting CUDA Graphs for the target model, which required substantial engineering effort and yielded no end-to-end speedups. This was because the target model kernels had large-enough workloads to keep the active GPU stream full while host code enqueued subsequent launches. As a result, the reduction in CPU launch overhead was completely hidden by the execution time of the target model's execution, rendering the optimization redundant in the overall inference pipeline. This was especially unfortunately given how much engineering was required to remove all host-side operations that would break the graph capture from the kernel launch while preserving kernel performance.

References

Cai, T., Li, Y., Geng, Z., Peng, H., Lee, J. D., Chen, D., and Dao, T. Medusa: Simple llm inference acceleration framework with multiple decoding heads, 2024. URL https://arxiv.org/abs/2401.10774.

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Ré, C. Flashattention: Fast and memory-efficient exact attention with io-awareness, 2022. URL https://arxiv.org/abs/2205.14135.

Dao, T., Haziza, D., Massa, F., and Sizov, G. Flash-decoding for long-context inference. https://pytorch.org/blog/flash-decoding/, October 2023. PyTorch Blog.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Ef-

ficient memory management for large language model serving with pagedattention, 2023. URL https://arxiv.org/abs/2309.06180.

Leviathan, Y., Kalman, M., and Matias, Y. Fast inference from transformers via speculative decoding, 2023. URL https://arxiv.org/abs/2211.17192.

Liang, W. and Liu, T. Large scale transformer model training with tensor parallel (tp). https://docs.pytorch.org/tutorials/intermediate/TP_tutorial.html, 2024. Accessed: 2026-06-08.

Luitjens, J. Cuda pro tip: Increase performance with vectorized memory access. URL https://developer.nvidia.com/blog/cuda-pro-tip-increase-performance-with-vectorized-memory-access/

Miao, X., Oliaro, G., Zhang, Z., Cheng, X., Wang, Z., Zhang, Z., Wong, R. Y. Y., Zhu, A., Yang, L., Shi, X., Shi, C., Chen, Z., Arfeen, D., Abhyankar, R., and Jia, Z. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ASPLOS '24, pp. 932–949. ACM, April 2024. doi: 10.1145/3620666.3651335. URL http://dx.doi.org/10.1145/3620666.3651335.

NVIDIA Corporation. Cuda library samples: cublas, 2020. URL https://github.com/NVIDIA/CUDALibrarySamples/tree/main/cuBLAS.

Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., and Liu, Y. Roformer: Enhanced transformer with rotary position embedding, 2023. URL https://arxiv.org/abs/2104.09864.

Wang, G., Lin, Y., and Yi, W. Kernel fusion: An effective method for better power efficiency on multithreaded gpu. In *2010 IEEE/ACM Int'l Conference on Green Computing and Communications Int'l Conference on Cyber, Physical and Social Computing*, pp. 344–350, 2010. doi: 10.1109/GreenCom-CPSCom.2010.102.

A. Appendix

A.1. Speculative Decoding Algorithm

Algorithm 1 (right) outlines the original speculative decoding algorithm proposed by (Leviathan et al., 2023).

Algorithm 1 Speculative Decoding Algorithm (from original Leviathan et al. paper)

Inputs: $M_p, M_q, prefix$.
▷ Sample γ guesses $x_{1,\ldots,\gamma}$ from M_q autoregressively.
for $i = 1$ **to** γ **do**
  $q_i(x) \leftarrow M_q(prefix + [x_1, \ldots, x_{i-1}])$
  $x_i \sim q_i(x)$
end for
▷ Run M_p in parallel.
$p_1(x), \ldots, p_{\gamma+1}(x) \leftarrow$
  $M_p(prefix), \ldots, M_p(prefix + [x_1, \ldots, x_\gamma])$
▷ Determine the number of accepted guesses n.
$r_1 \sim U(0,1), \ldots, r_\gamma \sim U(0,1)$
$n \leftarrow \min(\{i - 1 \mid 1 \le i \le \gamma, r_i > \frac{p_i(x)}{q_i(x)}\} \cup \{\gamma\})$
▷ Adjust the distribution from M_p if needed.
$p'(x) \leftarrow p_{n+1}(x)$
if $n < \gamma$ **then**
  $p'(x) \leftarrow norm(\max(0, p_{n+1}(x) - q_{n+1}(x)))$
end if
▷ Return one token from M_p, and n tokens from M_q.
$t \sim p'(x)$
return $prefix + [x_1, \ldots, x_n, t]$

A.2. Kernel Implementation Details

A.2.1. Embedding

The embedding is a memory-bound row-gather: out$[n,:] = W_{emb}[\text{ids}[n],:]$. We launch one block per output token (gridDim.$x = N$) with 128 threads each. threads stride across the hidden dimension, each copying $H/128 \approx 28$ `half` elements. Consecutive threads read consecutive addresses of a single weight row, so every warp issues a fully coalesced 256-byte load.

A.2.2. RMSNorm

RMSNorm normalizes each token independently: for token x_i, $RMS(x_i) = \sqrt{\epsilon + \frac{1}{h}\sum_{j=1}^{h} x_{i,j}^2}$, and the normalized output $\hat{x}_i = \gamma_i * \frac{x_i}{RMS(x_i)}$. We launch one thread-block per token and reduce across the hidden dimension. Each thread accumulates a partial sum with a warp-level reduction via `__shfl_down_sync`, followed by a token-level reduction that stages the per-warp partials through a small `__shared__` array and reduces them in the first warp of the block. The resulting `rsqrt` of the mean-square is broadcast through shared memory and applied in a second pass over h. Both passes cast the `fp16` buffers to `float4`, so each thread loads and stores 128 bits (8 `half`) per instruction. Adjacent threads move adjacent `float4` elements, so efficient coalescing is realized.

A.2.3. ATTENTION

Attention layers contains several distinct kernels. The QKV and output projections are implemented as cuBLAS Tensor-Core GEMMs, accumulating in `fp32`. The bulk of the attention layer is spent in a flash-attention kernel, which blocks over (sequence length, query-head, batch) with 128 threads per block. Q, K, and V tiles are held in `__shared__` memory, with the K and P (score) tiles overlaid in a `union` to save space, and the P tile padded to avoid bank conflicts for the WMMA load. The QK^T and PV GEMMs use the GPU's tensor core via `nvcuda::wmma` with $16\times16\times16$ fragments. The softmax is computed flash-attention style, so the KV values are streamed tile-by-tile without materializing the full score matrix. Qwen2.5 models use grouped-query attention, so each query head Q_i is mapped to it's respective KV head (Q_i/num_kv_heads). RoPE is fused into our custom KV write kernel (applied to K in place), and to flash attention (applied to Q in shared memory before the QK^T GEMM). HBM accesses of Q/K/V use `int4` (128-bit) vectorized accesses for coalescing.

Because decode operates on shorter sequence lengths, the workload becomes bandwidth bound on streaming the KV cache. We address this with a separate `decode_attn_kernel`, which blocks along the KV cache to keep occupancy high. The $16\times16\times16$ tensor core fragments are also completely underutilized, so individual threads instead compute `fp32` QK dot products. Because the target model may reject some tokens, the sequence length is unknown at compile time, so at runtime workloads are mapped to a pre-compiled distinct kernel for up to 8 tokens.

A.2.4. SWIGLU

The Qwen2.5 model family uses a SwiGLU MLP with no bias terms: $W_{down}(\text{SiLU}(W_{gate}(x)) \cdot W_{up}(x))$. The GEMMs are implemented as cuBLAS tensor-core GEMMs, and our custom kernel executes the elementwise multiply and activation $\text{silu}(\text{gate}) \cdot \text{up}$. It uses a grid-stride loop over the whole $[B * S, I]$ set of intermediate values, with each thread processing one `float4` (8 `fp16`) per step and computing SiLU in fp32. Accesses are coalesced with consecutive threads accessing consecutive data, and we fuse SiLU with the elementwise multiply in a single `swiglu_act_kernel` to avoid a separate HBM round-trip for the activations.

A.2.5. KV CACHE MANAGEMENT

Speculative decoding entails a more complex management of the KV cache than basic inference workloads. For both the target and draft models M_p and M_q, we track a `cache_pos` variable that indicates what KV values have been written to the cache. After both models complete the prefill phase on some prompt of length S, M_q will draft γ new tokens, updating its `cache_pos` to $S + \gamma$. M_p then verifies those drafted tokens in parallel, producing the bonus token, and update its `cache_pos` to $S + \gamma + 1$. However, M_p often accepts only $n < \gamma + 1$ tokens. In this case, we roll back M_p's `cache_pos` variable to position $S + n$, and pass $S + n$ as a variable via MPI for the draft model to also update its `cache_pos`. When the models perform their next forward pass, the updated variable ensures new KV values will overwrite the rejected tokens in memory.

B. MPI Communication Benchmarks

We benchmarked the MPI communication overhead between two Quadro RTX 6000 GPUs on a single node of the Stanford HPCC. The benchmarks measure the blocking transfer latency of the payloads with $\gamma = 4$ and a vocabulary size of $152,064$, using `MPI.Wtime()` averaged over 30 trials after 5 warmup iterations. Note that we also collected MPI benchmark data across a variety of configurations but all resulted in essentially the same performance results.

In the data, we see immediately that the communication time is dominated by the draft-to-target payload, which on average is 1.45 MiB. This payload is much larger than the corresponding target-to-draft payload, as the draft model must send its full token distribution to the target for each of its γ predictions, while the target model only sends back the number of accepted tokens and the index of the bonus token. The draft-side latency (which comes from the GPU-to-host Device-to-Host copy and the MPI send) averages **0.58 ms**, with D2H taking 0.39 ms at 3.9 GB/s and the MPI send taking 0.19 ms at 8.0 GB/s. The target-side latency (i.e. the MPI recv and Host-to-Device copy) averages **0.92 ms**, with MPI recv taking 0.68 ms at 2.2 GB/s and the H2D copy taking 0.24 ms at 6.5 GB/s. In contrast, the target-to-draft synchronization message is negligible, taking less than **10 microseconds** as a result of its miniscule 16-byte size. See Figure 3 for a visual breakdown.

Across all stages, a full speculative iteration incurs a mean communication latency of **1.51 ms**, which corresponds to an effective end-to-end bandwidth of 1.0 GB/s. This communication budget represents a theoretical limit of 650 iterations per second if GPU compute were free. Because the target 7B forward pass takes around 30 ms, the 1.51 ms communication overhead is minor and does not bottleneck the pipeline.

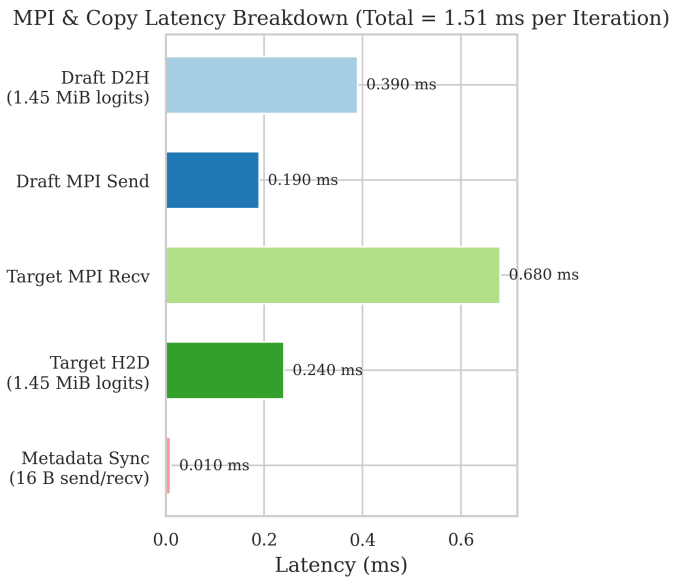

Figure 3. Latency breakdown of MPI across all speculative decoding sub-stages.